

Universidad Nacional de Colombia

Sede Bogotá, Facultad de Ingeniería

Asignatura: Inteligencia Artificial y Minirobots

Tema: Ejercicios Redes Neuronales

Elaborado por: Nicolas Rodriguez Cardenas , David Santiago Pina Jaramillo

- 1) Con base en el programa Clase_NNA3, desarrolle dos redes neuronales con 2 capas oculta, una que aprenda a reconocer una compuerta NAND y la otra aprenda a reconocer una compuerta XOR.
- 2) Con base en la librería tensorflow, descargue el data set fashion MNIST. Haga una clasificación de prendas de vestir. Explique cada una de las funciones y las principales instrucciones, saque conclusiones.
- 3) Consiga un data set de cualquier tipo, puede ser de <https://www.kaggle.com/datasets>, estudie sus características (features) y su rótulo. Diseñe una red neuronal y haga ejemplos con base en los pesos aprendidos.

Desarrollo

- 1) Partimos del programa Clase_NNA3, que es una red neuronal con dos capas ocultas implementada en Python con NumPy. Lo adaptamos para que aprendiera dos compuertas lógicas distintas.

Mantuvimos la arquitectura: 2 entradas, dos capas ocultas con 2 neuronas cada una, y una salida (1 neurona). Cambiamos los datos de entrenamiento: para la compuerta NAND usamos como entradas las cuatro combinaciones (00,01,10,11) y como salidas [1,1,1,0]; para la XOR usamos las mismas entradas y salidas [0,1,1,0].

La red al principio tiene pesos al azar, así que al darle una entrada (por ejemplo, 0,0) produce un número cualquiera entre 0 y 1. Luego comparamos con lo que debería dar (1 para NAND) y se calcula el error. Con la retropropagación, ese error se “devuelve” hacia atrás y se ajustan los pesos para que la próxima vez el error sea menor. Se repite esto en 10 000 épocas hasta que la red aprende la tabla de verdad. Resultado: La red para NAND aprendió perfectamente (error casi cero). La red para XOR también aprendió, aunque a veces cuesta más porque XOR no es linealmente separable; con 10 000 épocas logró acertar en todos los casos. Al final mostramos las predicciones redondeadas y comparamos con lo esperado.

- 2) El código implementa un modelo de red neuronal para clasificar imágenes del dataset Fashion MNIST, que contiene 60,000 imágenes de entrenamiento y 10,000 de prueba de prendas de vestir en 10 categorías. Primero, carga y

explora los datos, normaliza los píxeles dividiendo entre 255 para que estén en el rango [0,1] y define los nombres de las clases. Luego construye una red secuencial con una capa Flatten que aplanan las imágenes 28×28 en vectores de 784 dimensiones, una capa densa oculta con 128 neuronas y activación ReLU, y una capa de salida con 10 neuronas y activación softmax para obtener probabilidades por clase. El modelo se compila con el optimizador Adam, la función de pérdida `sparse_categorical_crossentropy` y la métrica de precisión. Se entrena 10 épocas, utilizando el conjunto de prueba como validación. Finalmente, se evalúa el modelo en el conjunto de prueba y se muestran las predicciones para las primeras cinco imágenes junto con sus etiquetas reales.

- 3) Para este ejercicio utilizamos el dataset iris, el cual contiene información sobre características de flores, como lo son la longitud y ancho de pétalos y sépalos, el cual tiene como objetivo clasificar las flores en tres especies diferentes.
Los datos fueron en conjuntos de entrenamiento (80%) y prueba (20%), luego se aplicó la normalización de los datos utilizando `standardscaler` para así mejorar el desempeño de la red neuronal.

También implementamos una red neuronal multicapa (`MLPClassifier`) con dos capas ocultas de 16 y 8 neuronas respectivamente. Se utilizó la función de activación ReLU y para entrenar y optimizar el modelo utilizamos Adam.

El desempeño del modelo lo evaluamos utilizando las métricas de precisión, matriz de confusión y reporte de clasificación, lo que nos permitió analizar la capacidad del modelo para clasificar de manera correcta las especies de flores.